

QF-AgentOS Architecture

QF-AgentOS — An honest benchmark and evidence harness for quantum finance

QF-AgentOS contributors

Version 0.1.0 — 2026-07-16

Contents

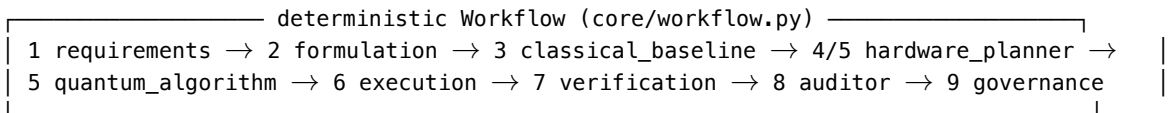
High-level flow	1
The agent contract	1
The three formulations (fair comparison)	2
Backends (backends/)	2
Safety model (core/policy.py)	2
Configuration & observability	3
Governance (governance/)	3
Surfaces	3
Extending	3

QF-AgentOS turns a financial optimisation problem into a verified, evidence-backed decision about *whether and how* to use quantum computing. This document describes the runtime, the agent contract, the backend abstraction, and the safety model.

High-level flow

ProblemSpec (Finance IR)

|
•



|
•

RunContext.state (typed PipelineState) + EvidenceBundle (manifest, report, model card)

The workflow is a fixed, ordered list of typed steps. Determinism is a feature: the same spec + seed produce the same evidence_digest. Steps are **isolated** — a failing agent is recorded as a StepError and the run continues so partial evidence is still emitted.

The agent contract

Each agent is Callable[[RunContext], str]. It reads and writes the **typed** blackboard ctx.state (core/state.py::PipelineState) — never a stringly-typed dict — and returns a one-line human summary. Artifacts exchanged between agents are Pydantic models in core/artifacts.py, so the inter-agent contract is mypy-checked and serialises cleanly into the evidence bundle.

#	Agent	Reads	Writes
1	requirements	spec	requirements
2	formulation	spec	formulations
3	classical_baseline	spec	classical_lp, classical_milp
4/5	hardware_planner	spec	instance, qubo, hardware_plan

#	Agent	Reads	Writes
5	quantum_algorithm	hardware_plan	quantum_selection
6	execution	instance, qubo, hardware_plan	instance_milp, instance_qubo_exact, instance_sa, instance_qaoa, qaoa_raw
7	verification	all results	verification, reproducibility
8	auditor	verification + results	audit
9	governance	everything	bundle

The three formulations (fair comparison)

The same economic problem is expressed so classical and quantum are compared like-with-like:

- **Continuous LP** — theoretical lower bound on cost.
- **Binary MILP** — the *fair* classical comparator (scipy/HiGHS, exact).
- **QUBO** → **Ising** — a reduced “research instance” for the quantum backend.

The QUBO encodes the coverage requirement as a proper inequality ($\text{cov} \geq R'$) using binary **slack bits**, with an **adaptive penalty** that guarantees the QUBO ground state respects coverage. Concentration and HQLA are *not* encoded — they become **verification-only** constraints. The Verification agent re-checks every decoded quantum solution against the *full* constraint set, so relaxation artifacts are caught, not hidden.

Backends (backends/)

All QUBO solvers implement the QuboSolver protocol (backends/base.py):

```
class QuboSolver(Protocol):
    name: str
    kind: str # classical | heuristic | quantum
    requires_credentials: bool
    def is_available(self) -> tuple[bool, str]: ...
    def solve(self, qubo: Qubo, config: QuboRunConfig) -> QuboSolution: ...
```

The registry is the single source of truth for discovery and instantiation:

backend	kind	credentials	notes
qubo_exact_optimum	classical	no	brute force ($\leq$ brute_force_qubit_limit)
simulated_annealing	heuristic	no	multi-restart SA
qaoa_sim	quantum	no	Qiskit statevector QAOA (default path)
qaoa_pennylane	quantum	no	PennyLane alternative simulator
qaoa_ibm	quantum	yes	IBM Runtime; optimise on sim, sample on QPU
dwave_hybrid	quantum	yes	D-Wave Leap hybrid sampler

Hardware backends are inert without their SDK + credentials (they raise `BackendUnavailableError`) and are gated behind autonomy **L3** + approval.

Safety model (core/policy.py)

Autonomy levels L0–L4 gate every side-effectful action:

Action	Min level	Extra gate
explain	L0	—
plan	L1	—
run_simulator	L2	—

Action	Min level	Extra gate
run_paid_qpu	L3	budget + human approval
recommend_production	L4	human approval

The gates have real call sites: the Execution agent authorises `RUN_SIMULATOR/RUN_PAID_QPU` before running a backend (chosen by `requires_credentials`), and the Auditor authorises `RECOMMEND_PRODUCTION`. The system never places trades, moves money, runs paid QPUs without approval, or claims quantum advantage without passing deterministic verification.

Configuration & observability

- `core/config.py` — environment-driven Settings (prefix `QF_`); credentials as `SecretStr` (never logged or serialised).
- `core/observability.py` — structured logging correlated by `run_id`, with an optional JSON format for ingestion.

Governance (governance/)

- `report.py` — builds the manifest, technical report, and model card, plus a deterministic evidence_digest (over decision-relevant content, excluding timestamps) for reproducibility checks.
- `store.py` — file-based experiment registry (`index.jsonl` + per-run bundle); swappable for a database behind the same interface.

Surfaces

- **CLI** (`cli.py`): `solve`, `explain`, `plan`, `skills`, `runs`, `serve`, `version`.
- **REST API** (`api.py`): `/healthz`, `/backends`, `/skills`, `/solve`, `/runs`.
- **SDK**: `from qf_agentos import load_spec, solve`.

Extending

See [CONTRIBUTING.md](#) for how to add a Quantum Skill, a backend, or a new problem family.